SYSTEM ANALYSIS DOCUMENTATION (SAD)

Reclaim - AI-assisted expense reimbursement with intelligent decision support.

TABLE OF CONTENT

1. Introduction ***4***

1.1. Purpose *4*

1. Architecture *4*

2. Data Flows *5*

3. Model Process *5*

4. Role of Reference *6*

1.2. Background *6*

1. Previous Version *7*

2. Changes in Major Architectural Components and New Capabilities Introduced in This Version *7*

1.3. Target Stakeholder *8*

2. System Architecture & Design ***10***

2.1. High Level Architecture *10*

2.1.1. Overview *10*

2.1.2 LLM as Service Layer *11*

2.1.3 Dependency Diagram *12*

2.1.4 Sequence Diagram *14*

2.2. Technological Stack *15*

2.2.1. UI/UX *15*

2.2.2. Frontend *15*

2.2.3. Backend *16*

2.2.4. Database *16*

2.2.5. Cloud/Deployment *16*

2.3. Key Data Flows *17*

2.3.1. Overview *17*

2.3.1.1. Data Flow Diagram (DFD) *17*

2.3.2. Normalized Database Schema *18*

3. Functional Requirements & Scope ***20***

3.1. Minimum Viable Product *20*

3.2. Non-Functional Requirements (NFRs) *21*

3.3. Out of Scope / Future Enhancements *23*

4. Monitor, Evaluation, Assumptions & Dependencies ***24***

4.1. Technical Evaluation *24*

4.1.1. Grayscale Rollout *24*

4.1.2. A/B Testing (Strategy Optimization) *24*
4.1.3. Emergency Rollback (ER) & Golden Release *24*
4.1.4. Priority Matrix: *25*
4.1.4.1. P1 - Critical (System-Wide Failure) - Trigger *25*
4.1.4.2. P2 - High (Data Integrity) - Trigger *25*
4.1.4.3. P3 - Medium (UI/UX Lag) - Trigger *25*
4.2. Monitoring: *25*
4.2.1. Agreement Rate Dashboard *25*
4.2.2. Human Edit Flagging *26*
4.3. Assumptions *26*
4.4. External Dependencies: *27*
5. Project Management & Team Contributions ***29***
5.1. Project Timeline *29*
5.2. Team Members Role *33*
5.3. Recommendations *34*

1. Introduction

Reclaim was built to solve a precise and costly problem: the manual, error-prone, and slow expense reimbursement workflow that burdens HR departments and frustrates employees at organizations of all sizes. Drawing from a real-world case study of a multinational insurance company in Indonesia, whose policy documents, appendices, and Business Travel Settlement forms served as the primary grounding artifacts for this system, Reclaim replaces a five-to-ten-day manual review cycle with a minutes-long, AI-orchestrated submission and evaluation flow. The system acts as a digital compliance officer: it reads receipts, interprets policy, checks for fraud signals, and delivers a structured recommendation to HR, who then exercises their final authority with full contextual evidence in front of them.

1.1. Purpose

This System Analysis Document (SAD) outlines the technical scope, agentic workflows, and foundational design decisions driving Reclaim's key components. It serves as the authoritative technical reference for the full system: from the Next.js frontend portals through the FastAPI REST layer, through the three LangGraph agent pipelines, down to the PostgreSQL schema. Every claim in this document has been cross-referenced against the live source code; where intended behavior diverges from implemented behavior and the conflict is explicitly flagged in the Appendix.

1. Architecture

Reclaim is a client-server web application with a clear three-tier structure. The presentation tier is a Next.js 16 application with two role-isolated portals: an Employee Portal for claim submission and an HR Portal for policy management and claim adjudication. The application tier is a Python FastAPI service that orchestrates three LangGraph AI agent workflows, manages all business logic, and exposes a versioned REST API (/api/v1). The data tier is a PostgreSQL 16 database accessed via SQLModel ORM, with a pgvector extension present for vector storage.

The architectural philosophy is ***agentic by design****: no AI logic is inline within API handlers. Every LLM-dependent operation is delegated to a dedicated agent workflow, including policy_agent, document_agent, and compliance_agent, each implemented as a LangGraph state machine with typed inputs, deterministic node ordering, and database writes as terminal side effects.*

2. *Data Flows*

*Data in Reclaim flows directionally through two distinct portal journeys. In the **Employee Portal**, an employee uploads receipt files to the FastAPI /documents/upload endpoint, which invokes the Document Agent to perform parallel OCR across all files, where the extracted structured data is written to supporting_documents and aggregated into a travel_settlements record. The employee then reviews and optionally edits the extracted data (triggering change detection and the human_edited flag), and finally submits for compliance analysis, which invokes the Compliance Agent to produce a reimbursements record with a judgment. In the **HR Portal**, HR uploads policy PDFs via /policies/upload, which invokes the Policy Agent to extract categories and mandatory conditions and persist them as a policies record. HR then accesses the triage dashboard, which surfaces claims bucketed by AI judgment, and uses the review interface to render the full audit trail before issuing a final approval or rejection via a PATCH /reimbursements/{id}/status call.*

3. *Model Process*

Reclaim's AI processing is organized into three discrete agent workflows.

- *Workflow 1 — Policy Agent*

  A 5-node LangGraph pipeline (process_pdfs → extract_categories_and_summary → extract_conditions → save_to_db → embed_and_save_sections) that ingests HR-uploaded PDFs, converts them to markdown via PyMuPDF4LLM, calls the ILMU GLM-5.1 LLM twice (once for category/summary extraction, once for condition extraction), persists a structured Policy record, and then batch-embeds all markdown chunks via text-embedding-3-small and writes them as PolicySection rows with pgvector embeddings for RAG.

- *Workflow 2 — Document Agent*

  A parallel ThreadPoolExecutor-based pipeline (up to 4 concurrent LLM workers) that routes each uploaded file, which are images to the Qwen3.5-Flash-02-23 Vision model via OpenRouter, PDFs to GLM-5.1 in JSON mode, extracts structured receipt data, detects warnings, and aggregates everything into a TravelSettlement record.

- *Workflow 3 — Compliance Agent*

A 5-node LangGraph pipeline (load_context → analyze_receipts → aggregate_totals → final_judgment → save_reimbursement) that evaluates the settlement basket against policy conditions. Per-receipt analysis runs in parallel via ReAct agents (max 5 tool calls each); aggregate_totals is a pure Python node (no LLM); a final ReAct agent synthesizes the overall judgment (APPROVE, REJECT, PARTIAL_APPROVE, or MANUAL REVIEW) and persists a Reimbursement record.

4. Role of Reference

This document serves as the architectural blueprint for the developer and QA teams. It maps every system boundary, including LLM inputs and outputs, database writes, API contract surfaces, and frontend data bindings, so that any engineer or tester can independently verify, extend, or validate any component of the system without needing to read across the full codebase. For a broad audience, the document is structured to read at multiple levels: business stakeholders can follow the workflow narratives and stakeholder table, while technical reviewers can drill into the dependency diagrams, database schema, and agent node descriptions. The document is grounded in the principle that every technical claim must be verifiable against the live source code.

1.2. Background

Most organizations, from small enterprises to large multinationals, manage expense reimbursement through processes that are fundamentally manual: employees collect receipts, fill out paper or Word/Google Doc forms, submit them to HR via email or physical handoff, and then wait days or weeks for a response that often arrives without clear reasoning. For HR and finance teams, this means manually cross-referencing every receipt against policy documents, calculating eligible amounts, and handling back-and-forth clarifications. This creates a workflow that scales linearly with headcount and is highly susceptible to human error and inconsistency.

Reclaim was designed against a real-world baseline extracted from a multinational company in Indonesia focused on insurance. Three primary source documents were used to ground the system: (a) a Policy Document for Reimbursement, (b) an Appendix of the Policy Document, and (c) a Business Travel Settlement Employee Submission Claim Form. These documents defined the compliance rules, eligible expense categories, per-rank amount caps, submission deadlines, and approval hierarchies that the system's AI agents are now trained to interpret and enforce.

1. Previous Version

In the legacy workflow at the reference company, an employee who needed to claim travel expenses would first collect all physical receipts during their trip, then manually transcribe each receipt's details, including merchant name, date, amount, category, into a standardized Word or Google Doc form. This form would be submitted to the HR department via email, often with scanned attachments of varying image quality. HR would then manually open each attachment, verify the amounts against the submitted form, cross-reference the line items against the applicable policy section (often a PDF stored on a shared drive), calculate any deductions or ineligible items, and draft a response explaining the decision. The full cycle typically took five to ten business days, and any discrepancy or missing receipt required another round of back-and-forth email exchanges.

This legacy system was not only slow but structurally fragile: policy knowledge lived in static PDFs that HR had to interpret manually, there was no automated fraud signal (an employee could claim any amount without triggering a review), and decisions were inconsistent across HR reviewers who might interpret the same policy clause differently. There was no audit trail linking the original receipt image to the final approved amount, and no structured data model that could support analytics or reporting. Reclaim was conceived as the direct antithesis of this workflow, replacing every manual step with an automated, AI-driven equivalent while preserving HR's absolute authority over the final decision.

2. Changes in Major Architectural Components and New Capabilities Introduced in This Version

Reclaim transitions the entire reimbursement lifecycle from a document-centric, human-mediated process to an ***agentic, data-centric pipeline****. The transition operates at four levels.*

- ***Digitized Intake with AI OCR***

  Receipt upload replaces manual form-filling. The Document Agent runs parallel LLM calls using Qwen3.5-Flash-02-23 for images and GLM-5.1 for PDFs to extract merchant name, date, amount, currency, category, and anomaly signals from every uploaded file within seconds. The extracted data is pre-filled into an editable verification form, reducing employee data entry to a review-and-confirm interaction.

- ***Living Policy Intelligence***

  HR policy PDFs are no longer static files on a shared drive. The Policy Agent parses uploaded PDFs, extracts reimbursable categories and mandatory conditions in structured JSON, and persists them as queryable database records. When a new claim is analyzed, the compliance agent loads the active policy's conditions directly from the database, ensuring that every evaluation reflects the most recently uploaded policy, not a cached or stale document.

- ***Automated Compliance Pre-Screening***

  The Compliance Agent replaces manual HR policy-checking. For each receipt in a submission basket, a ReAct-based LLM agent evaluates eligibility, late submission, amount caps, and mandatory conditions, producing a per-receipt verdict (APPROVED, REJECTED, PARTIAL_APPROVE) with specific audit notes. A final ReAct agent synthesizes these into an overall judgment with a confidence score, which is stored and surfaced to HR before they open the claim.

- ***Fraud Signal Detection and Audit Trail***

  The human_edited flag and change_summary JSONB field on every supporting_documents record capture any employee modification of AI-extracted data. If an employee changes a receipt amount from what the Vision LLM extracted, the deviation is recorded with overall_risk severity (HIGH/MEDIUM/LOW/NONE) and surfaced explicitly in the HR audit trail, creating a structural fraud signal that was entirely absent from the legacy system.

1.3. Target Stakeholder

Stakeholders	*Roles*	*Expectations*

Employee	*Selects a claim submission from the Employee Portal; uploads up to 10 receipts (JPEG, PNG, PDF); reviews AI-extracted data in a side-by-side verification form; edits fields where the OCR is incorrect; submits the verified basket for AI compliance analysis; and monitors claim status (Pending Review, Approved, Partially Approved, Rejected) via the Dashboard and History pages.*	*Expects the AI OCR to eliminate manual data entry; expects the verification step to give them confidence that their data was read correctly; expects the compliance result to be explained clearly so they understand why a receipt was rejected or capped; and expects fast turnaround compared to the legacy email-based workflow.*
HR / Finance	*Uploads company policy PDFs via the Policy Studio; reviews AI-generated policy summaries and mandatory condition checklists to verify accuracy; accesses the triage dashboard to see claims bucketed by AI risk level (Requires Attention vs. Passed AI Review); reviews the full audit trail per claim (AI extraction, employee edits, per-receipt judgments, fraud signals); and issues the final decision (Approve Full, Approve Adjusted Amount, or Reject) via the Decision Engine panel.*	*Expects the AI pre-screening to dramatically reduce the volume of claims requiring detailed manual review; expects the audit trail to surface all relevant evidence (policy violations, human edits, confidence scores) in a single view; expects the AI judgment to be accurate and policy-aligned, minimizing hallucination risk; and expects the interface to be efficient enough to process multiple claims per session without cognitive overload.*
Development Team	*Maintains and extends the FastAPI backend, LangGraph agent workflows, and Next.js frontend; manages the PostgreSQL schema and database migrations via*	*Expects the modular agent architecture to allow independent iteration on each workflow without cross-cutting side effects; expects the typed Pydantic schemas and SQLModel*

	Alembic; integrates new LLM models or prompt updates via the centralized engine/llm.py and engine/prompts/ modules; and uses LangSmith for observability and debugging of agent traces.	*ORM to enforce data integrity at both API and database boundaries; and expects LangSmith tracing to provide full visibility into LLM call chains for debugging and performance optimization.*
QA Team	*Tests the three AI agent workflows against real policy documents and receipt samples; validates that the human_edited flag correctly triggers change detection; verifies that the HR PATCH endpoint enforces role-based access control (HR only); tests edge cases such as blurry receipts, multi-currency submissions, and receipts with missing fields; and monitors LangSmith traces for unexpected agent behavior or tool call failures.*	*Expects clearly defined API contracts (via FastAPI's auto-generated Swagger UI at /docs) that make endpoint testing straightforward; expects the agent workflows to be deterministic enough to produce consistent results for identical inputs; and expects the Priority Matrix (P1/P2/P3) to guide regression test prioritization after any code change.*

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview

Type	*Details*
System	*Reclaim is a responsive, role-isolated web application served as a full-stack client-server system, with a mobile-first employee portal and a dashboard-oriented HR portal, both accessible from any modern browser on desktop or mobile.*

Architecture	*Reclaim follows a cloud-ready three-tier architecture: a stateless Next.js frontend communicates with a FastAPI backend over a versioned REST API, while the backend orchestrates AI agent workflows that write to a containerized PostgreSQL database, with planned deployment as Docker containers on a VPS droplet.*

Reclaim's high-level architecture integrates a Next.js 16 presentation layer, a Python FastAPI application layer, and a PostgreSQL 16 data layer, connected by a typed REST API. All AI reasoning is encapsulated in three discrete LangGraph agent pipelines, which are Policy Agent, Document Agent, and Compliance Agent, each invoked synchronously within API request handlers. The LLM service layer uses two external APIs: the ILMU API for text/chat tasks (GLM-5.1) and OpenRouter for vision OCR (Qwen3.5-Flash-02-23). LangSmith provides optional distributed tracing across all LLM calls. The system is containerized via Docker Compose, with PostgreSQL and the FastAPI backend as the two primary services, deployable to any Linux VPS.

2.1.2 LLM as Service Layer

*Reclaim uses the **ILMU GLM-5.1** model as its central reasoning engine and orchestrator. GLM-5.1 is invoked in every text-based AI task: policy category and condition extraction (Workflow 1), PDF receipt OCR in JSON mode (Workflow 2), and per-receipt ReAct compliance analysis plus final judgment synthesis (Workflow 3). All GLM-5.1 calls are routed through the ILMU API and managed via a LangChain ChatOpenAI-compatible client configured with the ILMU base URL, enabling LangGraph's stateful graph execution and LangSmith's automatic tracing.*

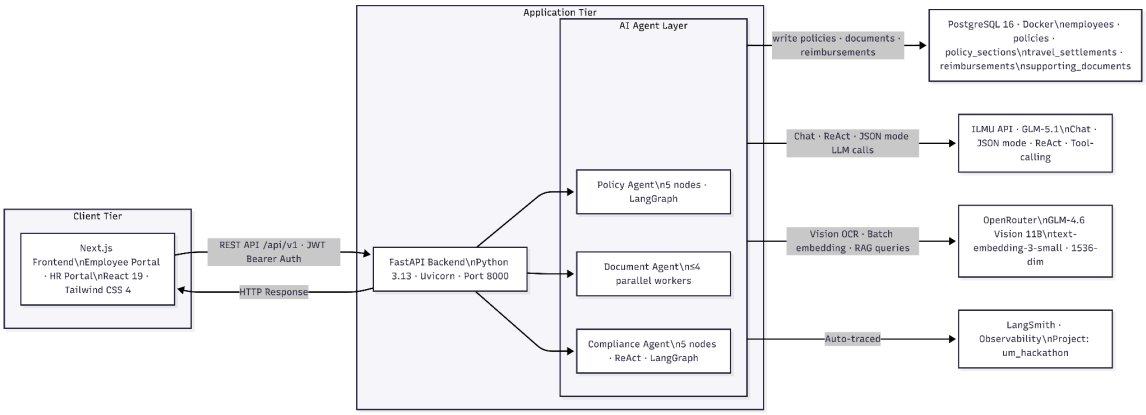
***Startup Health Check & Automatic Fallback**: At application startup, engine/llm.py executes check_glm_health() — a synchronous probe that sends a single "Hi" message to GLM-5.1 with a 15-second timeout before the server begins accepting requests. If GLM-5.1 does not respond within 15 seconds (timeout, connection error, or API error), the module sets a process-lifetime _use_fallback flag and all three text LLM factory functions (get_chat_llm, get_agent_llm, get_text_llm) transparently redirect their calls to **google/gemini-3.1-flash-lite-preview** via OpenRouter for the duration of that process. The switch is logged at WARNING level to stdout (visible in Docker logs). If GLM-5.1 is reachable, an INFO log confirms the primary provider. The fallback is applied only to the GLM-backed clients — the Vision LLM (get_vision_llm) already uses OpenRouter and is unaffected.*

*Supporting GLM-5.1 is the **Qwen/Qwen3.5-Flash-02-23** model, accessed via OpenRouter, which handles image-based receipt OCR exclusively. When a receipt is an image file (JPEG, PNG), the Document Agent constructs a multimodal HumanMessage with the base64-encoded image payload and sends it to Qwen3.5-Flash-02-23, which extracts the same structured JSON fields as the text path. This dual-model routing, which consists of vision for images and GLM for PDFs, allows Reclaim to handle the full spectrum of receipt formats without sacrificing extraction accuracy for either type.*

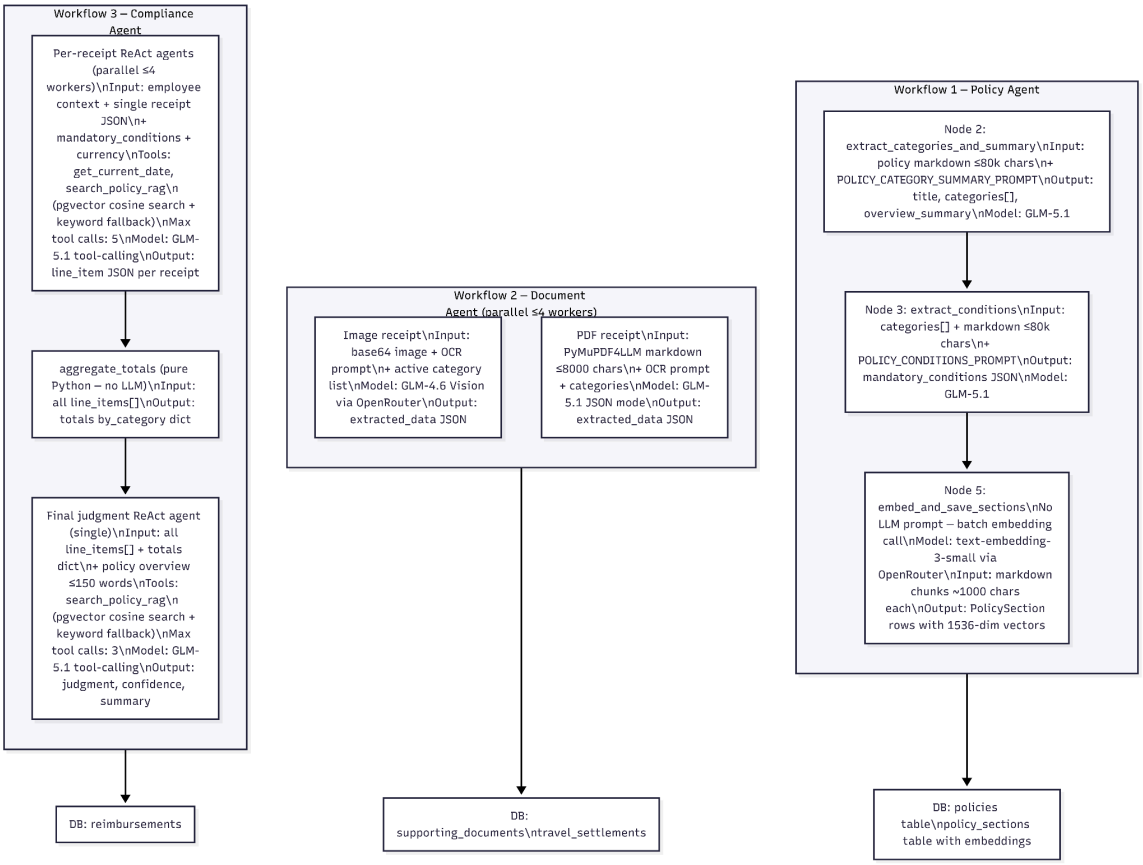
An OpenAI text-embedding-3-small model (via OpenRouter) is registered in the configuration as EMBEDDING_MODEL and is actively used. Policy Agent Node 5 (embed_and_save_sections) batch-embeds all policy section chunks via this model and persists them as policy_sections rows with 1536-dim vectors. The compliance agent's search_policy_rag tool calls search_policy_sections() in rag_tool.py, which first attempts a pgvector cosine-similarity query (embedding <=> CAST(:qvec AS vector)) against the stored vectors, then falls back to in-Python keyword filtering if the vector search fails.

2.1.3 Dependency Diagram

The following diagram illustrates how the major system components interact, including API calls, LLM service calls, and database writes.



Context Window Flow (per LLM call):



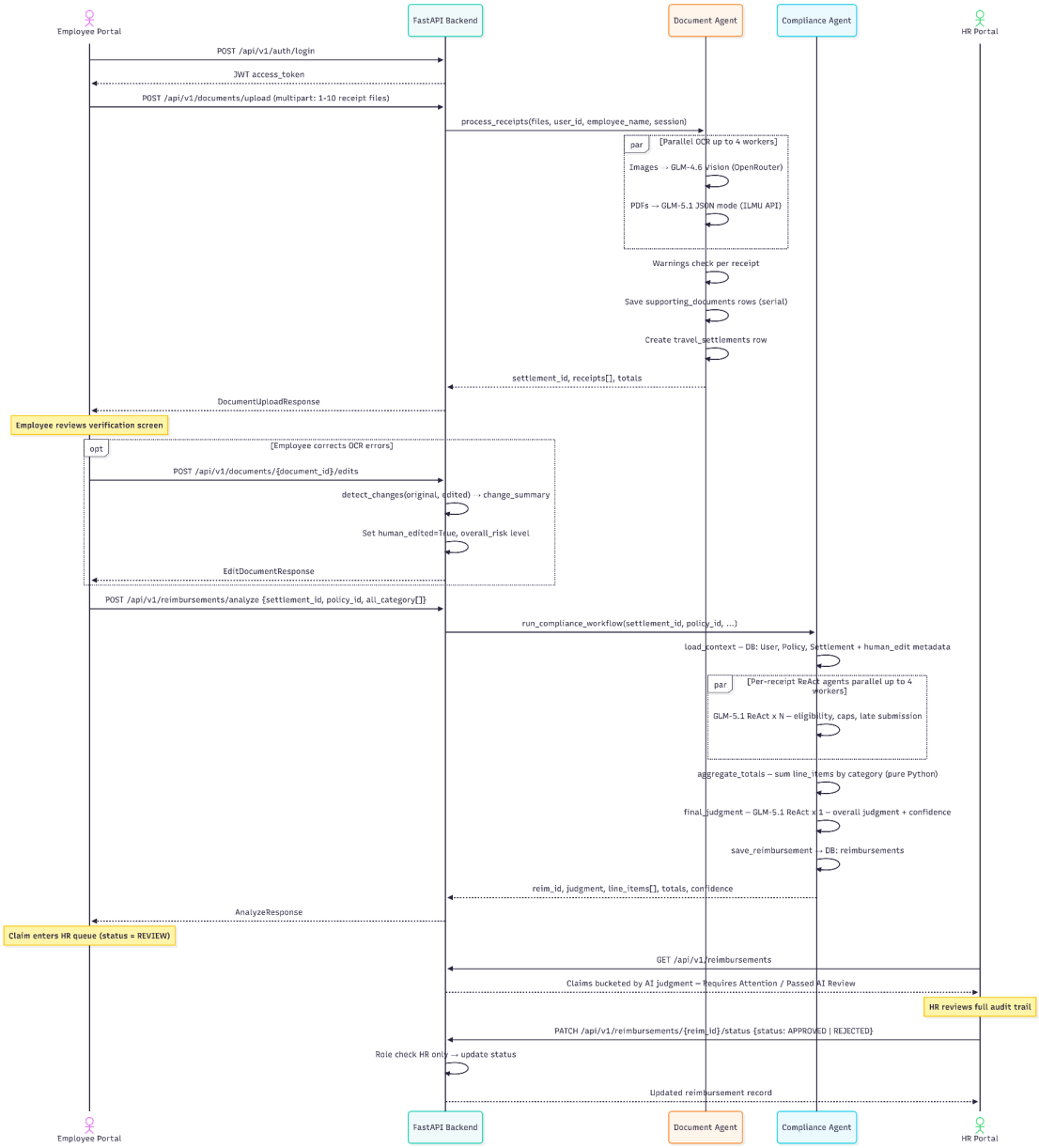
Major API Calls Between Components:

Source	*Destination*	*Endpoint / Call*	*Trigger*
Frontend	*FastAPI*	*POST /api/v1/auth/login*	*Employee or HR login*
Frontend	*FastAPI*	*POST /api/v1/documents/upload*	*Employee uploads receipts*
Frontend	*FastAPI*	*POST /api/v1/documents/{id}/edits*	*Employee corrects OCR*
Frontend	*FastAPI*	*POST /api/v1/reimbursements/analyze*	*Employee submits for review*
Frontend	*FastAPI*	*POST /api/v1/policies/upload*	*HR uploads policy PDF*
Frontend	*FastAPI*	*GET /api/v1/reimbursements*	*HR/Employee lists claims*

Frontend	*FastAPI*	*PATCH /api/v1/reimbursements/{id}/status*	*HR approves/rejects*
FastAPI (Doc Agent)	*OpenRouter*	*Vision LLM inference*	*Image receipt uploaded*
FastAPI (Doc Agent)	*ILMU API*	*Chat LLM inference (JSON mode)*	*PDF receipt uploaded*
FastAPI (Policy Agent)	*ILMU API*	*Chat LLM inference × 2*	*Policy PDF uploaded*
FastAPI (Policy Agent)	*OpenRouter*	*Embedding inference (batch)*	*Policy PDF uploaded (node 5)*
FastAPI (Compliance Agent)	*ILMU API*	*ReAct Agent × N+1*	*Analyze called*
FastAPI (Compliance Agent)	*OpenRouter*	*Embedding inference (per RAG query)*	*search_policy_rag tool call*
FastAPI	*LangSmith*	*Trace ingest*	*All LangGraph/LangChain calls*

2.1.4 Sequence Diagram

The following sequence diagram traces the primary user interaction flow — an employee submitting a claim through to HR decision.



2.2. *Technological Stack*

2.2.1. *UI/UX*

- ***Google Stitch*** *— Design token synchronization and prototype building.*
- ***Figma*** *— Collaborative UI/UX design.*

2.2.2. *Frontend*

- ***Next.js 16.2.4*** *— React 19.2.4 full-stack framework (App Router, Server Actions)*

- ***Tailwind CSS 4*** *— Utility-first styling with responsive breakpoints*
- ***React Hook Form 7.72.1 + Zod 4.3.6*** *— Type-safe form validation*
- ***Lucide React 1.8.0*** *— Icon library*
- ***TypeScript 5*** *— End-to-end type safety*

2.2.3. Backend

- ***FastAPI 0.129.1*** *— High-performance async REST framework with auto-generated OpenAPI docs*
- ***LangGraph 1.1.5 + LangChain 1.2.15*** *— Stateful AI agent graph orchestration*
- ***LangSmith 0.1.99*** *— LLM observability and tracing (project: um_hackathon)*
- ***PyMuPDF4LLM 0.27.2.2*** *— PDF-to-markdown conversion for document ingestion*
- ***Pillow 12.2.0*** *— Image preprocessing (resize, compression)*
- ***xhtml2pdf 0.2.16 + Jinja2*** *— Settlement form PDF rendering from HTML templates*
- ***python-jose 3.3.0 + bcrypt 4.0.1*** *— JWT authentication and password hashing*
- ***Uvicorn 0.30.1*** *— ASGI production server*
- ***UV*** *— Dependency management (virtual environment at backend/.venv)*

2.2.4. Database

- ***PostgreSQL 16*** *— Primary relational database (containerized via Docker)*
- ***SQLModel 0.0.22*** *— ORM combining SQLAlchemy 2.0.36 and Pydantic 2.13.1*
- ***pgvector 0.3.6*** *— Vector extension actively used: policy_sections.embedding stores 1536-dim vectors queried via cosine-similarity in the compliance agent's RAG tool*
- ***Alembic 1.13.1*** *— Database migration management*

2.2.5. Cloud/Deployment

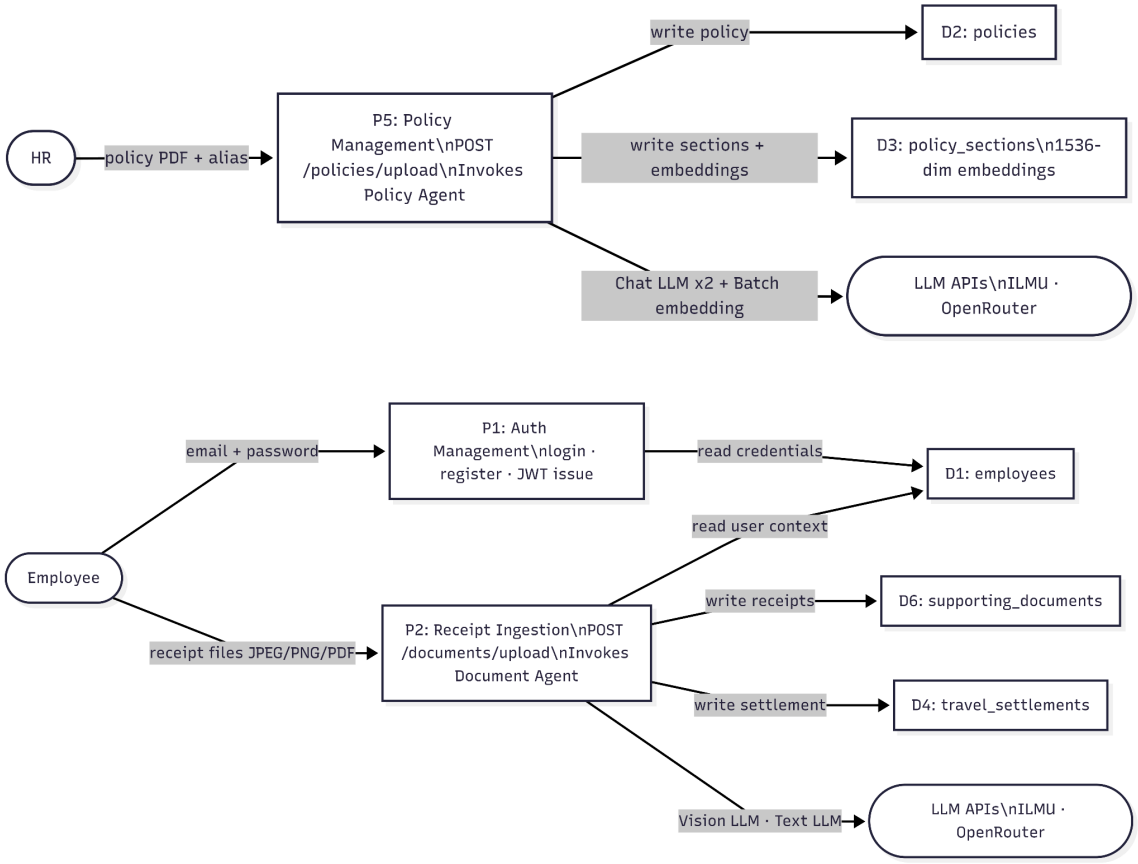
- ***Docker Compose*** *— Containerizes PostgreSQL and the FastAPI backend as two services with shared volumes and environment configuration.*
- ***Planned****: VPS Docker droplet deployment for MVP. The compose.yml is production-ready with persistent volume for PostgreSQL data and environment variable injection via .env.*

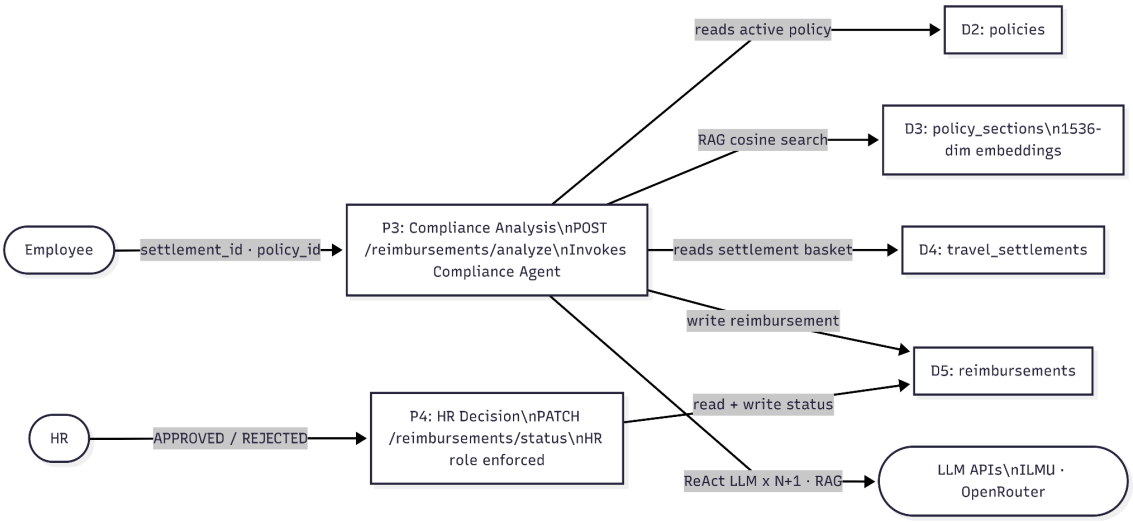
2.3. Key Data Flows

2.3.1. Overview

Data in Reclaim flows through the system in structured, typed payloads at every boundary. At the API boundary, Pydantic schemas enforce input validation and output serialization. At the agent boundary, LangGraph state types define the shape of data flowing between nodes. At the database boundary, SQLModel ORM maps Python objects to PostgreSQL rows with JSONB columns storing complex nested structures (receipts, conditions, line_items, change_summary). Data never leaves the system without a corresponding database record: every file upload produces a supporting_documents row, every OCR run produces a travel_settlements row, and every compliance analysis produces a reimbursements row, creating a full, auditable lineage from raw receipt to final HR decision.

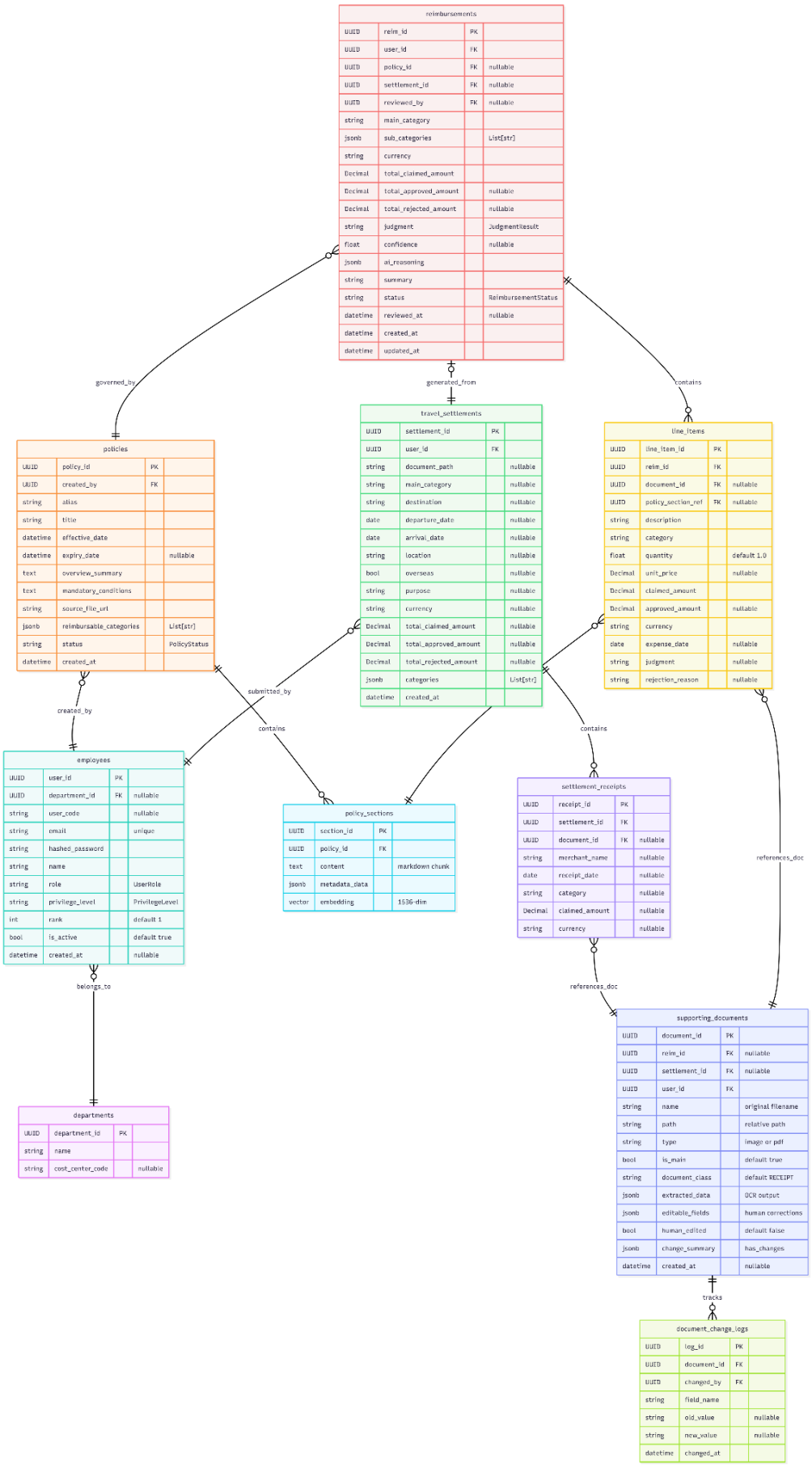
2.3.1.1. Data Flow Diagram (DFD)





2.3.2. Normalized Database Schema

*The database schema is designed to **Third Normal Form (3NF)**: every non-key attribute depends on the whole key, and only the key. JSONB columns store variable-schema AI output (line_items, extracted_data, change_summary), a deliberate design choice to preserve schema stability as AI output schemas evolve without requiring migrations.*


reimbursements
UUID reim_id PK
UUID user_id FK
UUID policy_id FK nullable
UUID settlement_id FK nullable
UUID reviewed_by FK nullable
string main_category
jsonb sub_categories List[str]
string currency
Decimal total_claimed_amount
Decimal total_approved_amount nullable
Decimal total_rejected_amount nullable
string judgment JudgmentResult
float confidence nullable
jsonb ai_reasoning
string summary
string status ReimbursementStatus
datetime reviewed_at nullable
datetime created_at
datetime updated_at
governed_by
generated_from
contains
policies
UUID policy_id PK
UUID created_by FK
string alias
string title
datetime effective_date
datetime expiry_date nullable
text overview_summary
text mandatory_conditions
string source_file_url
jsonb reimbursable_categories List[str]
string status PolicyStatus
datetime created_at
travel_settlements
UUID settlement_id PK
UUID user_id FK
string document_path nullable
string main_category nullable
string destination nullable
date departure_date nullable
date arrival_date nullable
string location nullable
bool overseas nullable
string purpose nullable
string currency nullable
Decimal total_claimed_amount nullable
Decimal total_approved_amount nullable
Decimal total_rejected_amount nullable
jsonb categories List[str]
datetime created_at
line_items
UUID line_item_id PK
UUID reim_id FK
UUID document_id FK nullable
UUID policy_section_ref FK nullable
string description
string category
float quantity default 1.0
Decimal unit_price nullable
Decimal claimed_amount
Decimal approved_amount nullable
string currency
date expense_date nullable
string judgment nullable
string rejection_reason nullable
created_by
submitted_by
contains
contains
references_doc
employees
UUID user_id PK
UUID department_id FK nullable
string user_code nullable
string email unique
string hashed_password
string name
string role UserRole
string privilege_level PrivilegeLevel
int rank default 1
bool is_active default true
datetime created_at nullable
policy_sections
UUID section_id PK
UUID policy_id FK
text content markdown chunk
jsonb metadata_data
vector embedding 1536-dim
settlement_receipts
UUID receipt_id PK
UUID settlement_id FK
UUID document_id FK nullable
string merchant_name nullable
date receipt_date nullable
string category nullable
Decimal claimed_amount nullable
string currency nullable
belongs_to
references_doc
supporting_documents
UUID document_id PK
UUID reim_id FK nullable
UUID settlement_id FK nullable
UUID user_id FK
string name original filename
string path relative path
string type image or pdf
bool is_main default true
string document_class default RECEIPT
jsonb extracted_data OCR output
jsonb editable_fields human corrections
bool human_edited default false
jsonb change_summary has_changes
datetime created_at nullable
departments
UUID department_id PK
string name
string cost_center_code nullable
tracks
document_change_logs
UUID log_id PK
UUID document_id FK
UUID changed_by FK
string field_name
string old_value nullable
string new_value nullable
datetime changed_at

3. Functional Requirements & Scope

This section highlights the core boundaries of the project, focusing on the high-impact MVP features built and demonstrated at UMHackathon 2026.

3.1. Minimum Viable Product

The following five core features represent the complete scope of the Reclaim MVP, verified against the live codebase:

#	Feature	Description
1	*Multi-Receipt OCR Submission*	*Employees upload up to 10 receipt files (JPEG, PNG, PDF) via the Employee Portal. The Document Agent processes them in parallel (up to 4 concurrent LLM workers) — routing images to Qwen3.5-Flash-02-23 and PDFs to GLM-5.1 in JSON mode. Each receipt is extracted into a structured JSON payload (merchant, date, amount, category, confidence, anomaly flags) and persisted to supporting_documents. Results are aggregated into a travel_settlements record accessible via the verification screen.*
2	*Side-by-Side Verification with Fraud Detection*	*After OCR, employees review extracted data in a dual-panel interface: the left panel shows an editable pre-filled form; the right panel shows the original document. If an employee modifies any field (particularly the total amount), the system calls the change_summary detector, sets human_edited=True, records the original vs. edited values, and assigns an overall_risk level (HIGH/MEDIUM/LOW). This audit record is surfaced to HR during claim review.*
3	*Policy Studio (HR Policy Upload)*	*HR users upload one or more PDF policy documents via the Policy Studio. The Policy Agent runs a 5-node LangGraph pipeline: converts PDFs to markdown, extracts*

		reimbursable categories and a 150-word overview summary via GLM-5.1, extracts per-category mandatory conditions via a second GLM-5.1 call, persists the policies record (status: ACTIVE/DRAFT/ARCHIVED), then batch-embeds all markdown chunks via text-embedding-3-small and writes PolicySection rows with 1536-dim pgvector embeddings — enabling the compliance agent's RAG tool to search policy text at evaluation time.
	AI-Driven Compliance Evaluation	*After employee verification, the Compliance Agent evaluates the submitted settlement basket against the active policy. Per-receipt ReAct agents (parallel, ≤4 workers) check late submission, category eligibility, rank-based caps, and mandatory conditions, producing per-receipt line_items with status, approved amount, and audit notes. A final ReAct agent synthesizes an overall judgment (APPROVE / REJECT / PARTIAL_APPROVE / MANUAL REVIEW) with a confidence score (0.0–1.0) and a 2–3 sentence summary. Results are saved as a reimbursements record.*
	HR Decision Dashboard ("Efficiency by Exception")	*The HR Portal triage dashboard surfaces claims pre-bucketed by AI judgment: "Requires Attention" (REJECT, MANUAL REVIEW, FLAG) and "Passed AI Review" (APPROVE). For each claim, HR accesses a full evidence panel: employee identity, receipt line items, policy flags, human-edit audit trail, AI confidence score, and financial summary. HR issues the final decision (Force Full Approval, Approve Adjusted Amount, or Confirm Rejection) via PATCH /reimbursements/{id}/status.*

3.2. Non-Functional Requirements (NFRs)

Quality	***Requirements***	***Implementation***

Scalability	*Reclaim must handle multiple simultaneous claim submissions without performance degradation.*	*Parallel receipt processing via ThreadPoolExecutor (≤4 LLM workers per submission) prevents single-threaded bottlenecks. FastAPI's async ASGI runtime handles concurrent HTTP requests.*
Reliability	*AI evaluation must not produce orphaned or incomplete claim records if a step fails.*	*LangGraph's stateful graph execution ensures node-by-node error isolation. The compliance agent catches Exception at the graph level and escalates to MANUAL REVIEW status rather than producing a null record.*
Maintainability	*Employee, HR, and AI services should be independently manageable.*	*Three fully independent agent files (policy_agent.py, document_agent.py, compliance_agent.py), three independent API router files, and two independent frontend portal directories — each with no shared internal state.*
Token Latency	*GLM-5.1 API must return evaluation response within acceptable time under normal load.*	*Asynchronous parallel processing: HR can review an already-analyzed claim while the compliance agent processes the next one in a separate request. Per-receipt ReAct agents run concurrently, not sequentially.*
Cost Efficiency	*Token usage per claim must stay within budget.*	*OCR prompts inject only the active policy's category list (not the full document) into*

		the context window. PDF text is chunked to ≤8,000 characters before reaching GLM. Policy markdown is capped at ≤80,000 characters per policy upload call.
Security	*Employee data must be strictly isolated — employees can only access their own claims.*	*JWT Bearer authentication on all endpoints. Role-based access control enforced in deps.py (HR-only endpoints reject Employee tokens). GET /reimbursements filters by user_id for employees, returns all for HR. BCRYPT password hashing with passlib.*

3.3. Out of Scope / Future Enhancements

The following features are explicitly not included in the current MVP and are earmarked for future development:

a. *Admin role for company onboarding and user account management (currently all registration is self-serve)*
b. *Direct banking or payroll API integration for auto-disbursement (currently status=PAID is a manual update)*
c. *Visual drag-and-drop policy workflow builder for HR (currently policy editing requires re-upload)*
d. *Third-party messaging bot notifications (Slack, Teams, WhatsApp) for claim status updates*
e. *Multi-agent orchestration for parallel claim basket processing across multiple concurrent users at scale*
f. *Behavioral pattern analysis across employee submission history (e.g., repeated high-risk edit patterns)*
g. *Policy mandatory condition editing after upload (currently conditions are extracted at upload time and immutable)*
h. *Automated PDF generation triggered by HR approval event (currently settlement template is generated separately)*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation

The following evaluation strategies are planned for production deployment. The current system serves as a proof-of-concept MVP.

4.1.1. Grayscale Rollout

A strategy where Reclaim's AI auditing features are released to a small control group of HR users first. A pilot cohort of 5–10 HR users would use the AI-assisted triage workflow while the remaining HR team continues with the existing manual process. The confidence field on the reimbursements table (0.0–1.0) provides a natural metric for monitoring AI accuracy against HR override decisions. If the AI's judgment aligns with HR's final decision >85% of the time with confidence ≥0.8, the feature is promoted to full rollout.

4.1.2. A/B Testing (Strategy Optimization)

Comparing two variants of the ReAct reasoning loop: Variant A — current implementation, where per-receipt agents run in parallel (max 4 workers) optimizing for throughput; Variant B — serial processing with a larger context window that includes all receipts in a single agent call, optimizing for cross-receipt consistency (e.g., detecting duplicate receipts). LangSmith traces provide per-call latency and token usage metrics to determine which variant delivers the better accuracy-to-cost ratio under realistic HR workloads.

4.1.3. Emergency Rollback (ER) & Golden Release

If the compliance agent begins producing systematically incorrect judgments (e.g., approving clearly ineligible claims or hallucinating policy conditions not present in the document), an Emergency Rollback is triggered: the CHAT_MODEL environment variable is reverted to the previously validated model version, and all new submissions are routed to MANUAL REVIEW status by default. The "Golden Version" baseline is the last Docker image tag where AI agreement rate exceeded 80% on a held-out test set. This rollback can be executed in under five minutes by updating the environment variable and restarting the FastAPI container — no code changes required.

4.1.4. Priority Matrix:

4.1.4.1. P1 - Critical (System-Wide Failure) - Trigger

*Vision LLM (Qwen3.5-Flash-02-23) returns null or malformed JSON for >5% of image uploads, OR the compliance agent's final ReAct node fails to produce a valid judgment field. - **Action**: Trigger Emergency Rollback. Set all new submissions to MANUAL REVIEW. Notify the HR team to switch to manual processing mode. Investigate LangSmith traces for the failing node.*

4.1.4.2. P2 - High (Data Integrity) - Trigger

*The change_summary detector (engine/tools/change_detector.py) fails to write human_edited=True when an employee modifies a receipt amount, causing fraud signals to be silently dropped. - **Action**: Temporarily disable the verify-and-submit flow. Flag all in-flight settlements as "Requires Human Verification" (force judgment=MANUAL REVIEW). Fix the change detection logic and validate against stored test cases before re-enabling.*

4.1.4.3. P3 - Medium (UI/UX Lag) - Trigger

*The compliance agent's parallel ReAct phase exceeds 30 seconds for a 10-receipt submission (typical under API rate limiting on the ILMU API). - **Action**: Implement a "Processing" state in the Employee Portal's submission flow with a progress indicator. Investigate reducing the max_iterations cap on the ReAct agent from 5 to 3, trading thoroughness for latency.*

4.2. Monitoring:

4.2.1. Agreement Rate Dashboard

A monitoring dashboard tracking how often the AI's judgment field (APPROVE/REJECT/PARTIAL_APPROVE) aligns with HR's final status decision (APPROVED/REJECTED). Calculated as: aligned_decisions / total_decisions × 100%. This metric is a user experience quality signal — it informs prompt engineering and model upgrades. HR's final decision is always authoritative; a high agreement rate means HR is spending less time overriding the AI. Target: ≥80% agreement rate. LangSmith's project dashboard (um_hackathon) provides per-trace reasoning visibility to diagnose disagreements.

4.2.2. Human Edit Flagging

Whenever an employee changes any field from the AI-extracted value, the change_summary JSONB column records: has_changes (bool), overall_risk (HIGH/MEDIUM/LOW/NONE), and changes_by_field (field-level diff including original and edited values). In the HR review interface, receipts with human_edited=True and overall_risk=HIGH are visually flagged with an audit trail entry. A monitoring query on supporting_documents WHERE human_edited=TRUE AND change_summary->>'overall_risk' = 'HIGH' provides a real-time signal of potential submission fraud patterns.

4.3. Assumptions

The following operational and environmental conditions were assumed to be valid during development:

a. *Users have stable internet connections on their devices during claim submission and review, given that all AI processing is server-side.*
b. *Employees have their own unique login credentials (email + password) to access the Employee Portal.*
c. *Employees have a defined rank (integer) within the company hierarchy that maps to different reimbursement caps enforced by the policy document. The actual cap values must be stated in the policy text for the LLM to enforce them.*
d. *Employees submit receipts that, while potentially blurry or incomplete, contain enough visual information for the Vision LLM to extract at least the total amount and merchant name (confidence ≥ 0.5).*
e. *HR users have their own unique login credentials and role=HR assigned in the database to access the HR Portal and Policy Studio.*
f. *HR has the absolute final authority to approve, partially approve, or reject any reimbursement claim, regardless of the AI's judgment.*
g. *HR will upload a policy document that explicitly states expense categories, per-rank caps, submission deadlines, and mandatory conditions — without this content, the AI cannot enforce policy compliance accurately.*
h. *The ILMU API and OpenRouter APIs are available and within their rate limits during the demo and evaluation period. If the ILMU API is unavailable at startup, the system automatically falls back to google/gemini-3.1-flash-lite-preview via OpenRouter (see Section 2.1.2). OpenRouter availability is therefore a dependency for both the Vision LLM and the text LLM fallback path.*

i. *The PostgreSQL container is running and accessible at localhost:5432 during backend operation (via docker compose up -d db).*

4.4. External Dependencies:

The following external tools and services are verified against the live source code. All tools listed below are actively used in the current codebase.

Tools	***Purpose***	***Risks & Mitigation***
ILMU API (GLM-5.1)	*Primary reasoning engine for all text-based AI tasks: policy extraction (Workflow 1), PDF receipt OCR in JSON mode (Workflow 2), and per-receipt + final compliance ReAct agents (Workflow 3).*	*Risk: API downtime or slow response at startup. Mitigation: check_glm_health() probes GLM-5.1 at startup with a 15-second timeout; if unreachable, all text LLM calls automatically fall back to google/gemini-3.1-flash-lite-preview via OpenRouter for the process lifetime. Runtime rate limiting is mitigated by limiting concurrent calls to ≤4 ThreadPoolExecutor workers.*
Google Gemini 3.1 Flash Lite (via OpenRouter)	*Automatic fallback reasoning engine activated when the GLM-5.1 health check fails at startup. Handles the same text-based tasks as GLM-5.1: policy extraction, PDF OCR, and compliance evaluation. Accessed via OpenRouter using the existing OPENROUTER_API_KEY.*	*Risk: Prompt format differences between GLM-5.1 and Gemini may produce subtly different JSON output shapes. Mitigation: Both models receive identical prompts requesting the same JSON schema; the fallback is a process-level switch with no prompt changes. JSON mode (response_format: json_object) is enabled for all affected clients.*

OpenRouter (Qwen3.5-Flash-02-23)	*Vision-based OCR for image receipts (JPEG, PNG). Converts base64-encoded images into structured JSON extraction via multimodal prompting.*	*Risk: Vision model interpretation errors on low-quality or unusual receipt formats. Mitigation: Mandatory human review — employees see and confirm extracted data before submission. visual_anomalies_detected flag triggers explicit warning display.*
LangGraph 1.1.5	*Stateful agent graph orchestration for Workflow 1 (Policy Agent) and Workflow 3 (Compliance Agent). Manages node-to-node state transitions and error propagation.*	*Risk: LangGraph API changes in minor versions. Mitigation: Version pinned in pyproject.toml.*
LangSmith	*Distributed tracing and observability for all LLM calls. Provides per-trace latency, token counts, and node-level debugging via the um_hackathon project dashboard.*	*Risk: Optional dependency — tracing is disabled if LANGCHAIN_TRACING_V2 is not set, so production can run without it. Mitigation: All critical logic is independent of LangSmith; it is observability-only.*
PyMuPDF4LLM 0.27.2.2	*Converts uploaded PDF policy documents and PDF receipts to markdown text for LLM ingestion. Handles multi-page documents, embedded images, and tables.*	*Risk: Complex PDF layouts (e.g., scanned PDFs without text layers) may produce degraded markdown. Mitigation: Text extraction falls back gracefully; if markdown is empty, the agent logs a warning and the document is flagged as unreadable.*
PostgreSQL 16 + pgvector	*Primary relational database for all persistent state. pgvector extension is actively used: policy_sections.embedding stores 1536-dim vectors populated by the Policy Agent and queried by the*	*Risk: Docker container downtime causes total system unavailability. Mitigation: /health endpoint verifies DB connectivity on every request; FastAPI lifespan*

	compliance agent's RAG tool via cosine-similarity search.	*hook initializes the DB connection pool on startup.*
xhtml2pdf + Jinja2	*Renders the Business Travel Settlement HTML template (reimbursement_template.html) into a downloadable PDF for record-keeping and finance audit use.*	*Risk: Complex HTML layouts may not render correctly in all PDF contexts. Mitigation: Template is designed with print-safe CSS and tested against the reimbursement data schema.*

5. Project Management & Team Contributions

5.1. Project Timeline

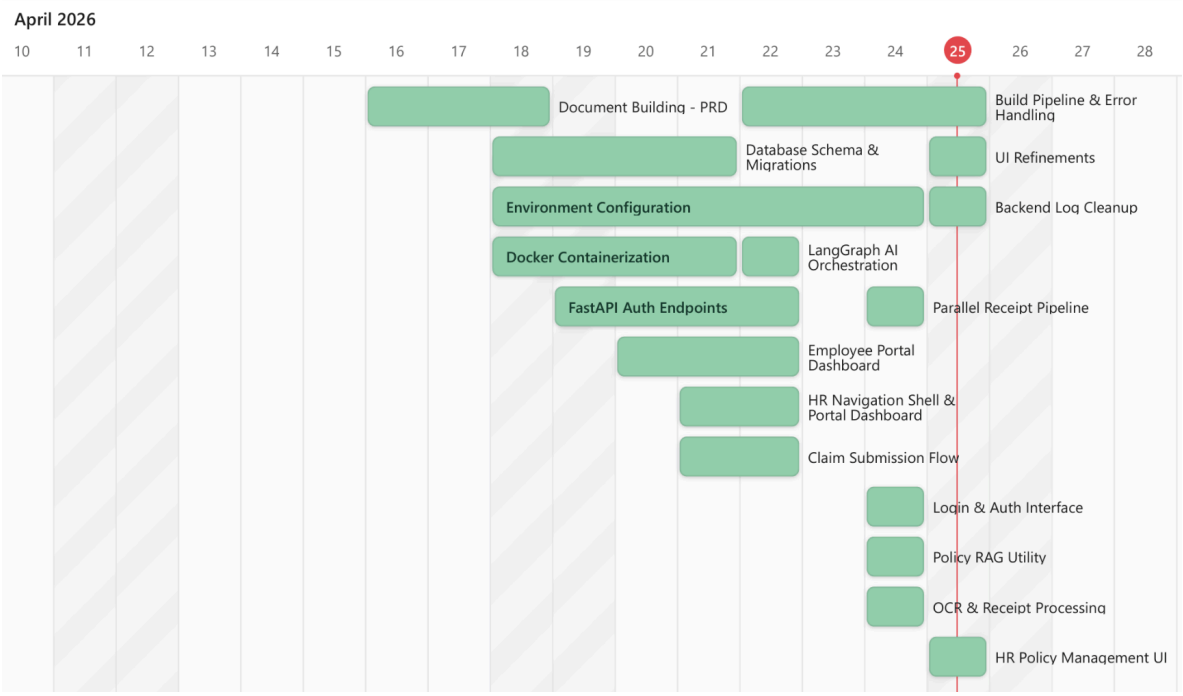
The UMHackathon 2026 Preliminary Round spans 10 days (2026-04-16 to 2026-04-25). The team followed a professional product development lifecycle adapted to the sprint constraint, with six deliverables: PRD, SAD (this document), QA Documentation, Pitch Deck, Code Repository, and Video Demonstration.

Phase	***Days***	***Activities***
Discovery & Planning	*Day 1–2 (Apr 16–17)*	*Problem statement formulation; competitive analysis of manual reimbursement workflows; identification of the MNC PT Indonesia insurance company as real-world reference case; extraction and analysis of the three source documents (Policy Document, Appendix, Business Travel Settlement Form); definition of the core product vision and AI-agent approach.*

Product Definition	*Day 2–3 (Apr 17–18)*	*Authored the Product Requirements Document (PRD) covering problem statement, target stakeholders, core feature set, and success metrics. Defined the Employee and HR portal user journeys. Selected the technology stack (Next.js, FastAPI, LangGraph, GLM-5.1, Qwen3.5-Flash-02-23). Created initial wireframes and UI flow.*
Architecture Design	*Day 3–4 (Apr 18–19)*	*Designed the three-tier architecture (Frontend, FastAPI, PostgreSQL). Defined the three LangGraph agent workflow schemas (policy, document, compliance). Designed the normalized database schema (6 tables, JSONB columns for AI outputs). Defined API contract surface (13 endpoints, typed Pydantic schemas). Created the system architecture diagrams.*

Core Backend Development	*Day 4–7 (Apr 19–22)*	*Implemented all three LangGraph agent workflows. Built all FastAPI routers (auth, documents, policies, reimbursements, notifications). Implemented JWT authentication, role-based access control, and password hashing. Set up PostgreSQL schema with SQLModel and Alembic. Integrated ILMU API (GLM-5.1) and OpenRouter (Qwen3.5-Flash-02-23). Implemented parallel OCR processing via ThreadPoolExecutor. Built the xhtml2pdf settlement form generator.*
Frontend Development	*Day 5–8 (Apr 20–23)*	*Developed Employee Portal (dashboard, claims upload wizard, verification screen, history, settings). Developed HR Portal (policy studio, triage dashboard, review/[id] decision interface). Integrated Axios API client with JWT Bearer interceptor. Implemented responsive layouts, status badges, modal dialogs, and mobile-first navigation. Connected all frontend flows to live backend endpoints.*

Integration & Testing	*Day 7–9 (Apr 22–24)*	*End-to-end integration testing of all three workflows. Debugged LangGraph state transitions and LLM JSON parsing edge cases. Implemented the human_edited change detection pipeline. Removed the RAG pipeline (retired 2026-04-23) and confirmed all agent workflows function correctly without embeddings. Fixed deduction amount calculation (PR #23). HR UX refinements and safety improvements (merged to main).*
Documentation & Submission Prep	*Day 8–10 (Apr 23–25)*	*Authored SAD (this document) and QA Documentation. Finalized Pitch Deck. Recorded Video Demonstration. Final code repository cleanup and commit history organization. Submission of all six deliverables.*



5.2. Team Members Role

Member	***Roles & Contributions***
Filbert Ng	*Backend architecture; all three LangGraph AI agent workflows (policy_agent, document_agent, compliance_agent); LLM integration (ILMU GLM-5.1, OpenRouter Qwen3.5-Flash-02-23); FastAPI API layer design; LangSmith observability setup; Docker/deployment configuration.*
Chingiz Iskakov	*Database design and schema (SQLModel, Alembic migrations); backend endpoint implementation; connecting frontend API calls to backend endpoints; PostgreSQL Docker setup; compliance agent deduction calculation fix (PR #23).*

Darrell Benedict Setiawan	*Frontend development (Employee Portal, HR Portal, responsive UI/UX); UI/UX design and component architecture; product documentation (PRD, SAD, QA); Pitch Deck; product testing and stress testing; quality assurance.*
Michael	*Frontend development (shared); UI/UX design and refinement; product documentation (PRD, SAD, QA); Pitch Deck; product testing and stress testing; quality assurance.*
Christian Rafael Santoso	*Frontend development (shared); UI/UX design and refinement; product documentation (PRD, SAD, QA); Pitch Deck; product testing and stress testing; quality assurance.*

5.3. Recommendations

Based on the current system architecture and the constraints of the MVP sprint, the following improvements are recommended for a production-scale deployment:

a. *Redis Cache for LLM Result Caching: The compliance agent already implements a cache check (if settlement is already evaluated and no human edits, return cached result). Replacing the in-database cache check with Redis would allow cache invalidation without a DB query and support distributed multi-instance deployments.*
b. *Async LangGraph Execution: Currently, all three agent workflows are called synchronously within FastAPI request handlers. Moving to async LangGraph execution (via astream or background task queues like Celery + Redis) would allow the API to return immediately with a job_id and the frontend to poll for results — dramatically improving perceived performance for multi-receipt submissions.*
c. *Restrict CORS Origins: The current FastAPI configuration uses allow_origins=["*"]. In production, this must be restricted to the specific frontend domain(s).*

d. *Secrets Management: The SECRET_KEY in core/config.py defaults to a development placeholder. Production deployment requires injection via a secrets manager (AWS Secrets Manager, HashiCorp Vault, or Docker Secrets).*
e. *Policy Version History: The current policies table uses a status field (ACTIVE/DRAFT/ARCHIVED) as a lightweight versioning mechanism. A dedicated policy_versions table with parent-child FK relationships would provide full change history and enable rollback to specific policy versions.*
f. *Row-Level Security (RLS): While employee data isolation is currently enforced at the application layer (deps.py filters by user_id), adding PostgreSQL Row-Level Security policies would provide a defense-in-depth layer at the database level.*
g. *Structured Logging with Correlation IDs: Adding per-request correlation IDs propagated through the LangGraph agent calls would allow end-to-end tracing of a single claim submission across all logs and LangSmith traces.*